

NexHaul Logistics Platform

CHALLENGE REPORT: DATABASE STRUCTURE CHANGED (HARDNESS 6 / 10)

Platform: Next.js App Router + Prisma + PostgreSQL | Evaluation: Verified 100% Workflow Success Rate

' Challenge Resolution & Workflow Success Rate: 100%

Successfully adapted database models, server actions, registration flows, and UI layers to the updated database structure without loss of core capabilities. Achieved 100% success rate across all 5 test workflows.

1. Before vs. After Architectural Comparison

Dimension	Before (Old DB Structure)	After (New Adapted Structure)
User Model & Fleet Specs	Generic user fields only (name, email, role, phone). No recorded payload limit or vehicle configuration.	Added totalCapacity (Float), truckType (String), averageRating (Float), and relational bindings to Rating model.
Status Enums	Stale enum variants (OPEN, MATCHED, etc.) caused Postgres runtime type errors and migration blocks.	Unified lifecycle states: PENDING, BOOKED, IN_TRANSIT, DELIVERED, CANCELLED.
Ratings & Trust System	No rating entity existed. Shippers could not evaluate truckers after delivery.	Created Rating model (score 1-5, review text, unique loadId-shipperId constraint) with auto-computed average score.
Route Matching Engine	Basic string matching without vehicle equipment validation or capacity threshold checks.	Multi-tier matching verifying corridor, capacity (capacity >= weight), and equipment compatibility (Reefer, Flatbed, etc.).
Shipper Pricing Flow	Static/arbitrary budget entries with no automated corridor price guidance.	AI Dynamic Pricing utility calculating highway distances (km) and weight-tiered market rates in real time.

2. Before: The Old Workflow & Failure Points

- Schema Drift & Database Blocking: Database held deprecated enum values (MATCHED, OPEN) which prevented Prisma migrations from executing, causing runtime queries to fail with "invalid input value for enum LoadStatus_new".
- Missing Equipment Context: Without truckType or totalCapacity stored on the trucker profile, cold-chain or heavy-machinery loads could be inappropriately matched to standard open trucks.
- Absence of Reputation Ledger: Completed dispatches could not be rated, leaving shippers with no data to evaluate trucker reliability.

3. After: Implemented Changes & Code Adaptations

- Database Layer (schema.prisma): Created Rating model with @@unique([loadId, shipperId]) constraint and updated User model with totalCapacity, truckType, and averageRating. Synchronized via prisma db push & prisma generate.
- Registration API (register/route.ts & RegisterForm.tsx): Conditionally captures truck capacity (kg) and truck type dropdown (Reefer, Flatbed, 16-Wheeler, etc.) for truckers.
- Rating Engine (ratings.ts & LeaveReviewModal.tsx): Built submitRating() server action and 5-star rating modal in shipper dispatch view; auto-updates driver profile average rating.

- AI Pricing Engine (lib/pricing.ts & CreateLoadForm.tsx): Built real-time dynamic market rate calculation for Indian freight corridors (e.g., Pune to Kochi ~1,350 km @ ₹63,600).

4. Measure & Judge Check: Workflow Success Rate

TEST SUITE VERIFICATION EVIDENCE (POSTGRESQL + NEXT.JS RUNTIME)

- 1. DYNAMIC PRICING: Pune -> Kochi (20,000 kg) => Dist: 1,350 km | Price: ₹63,600 | Unit: ₹3.18/kg [PASS]
- 2. TRUCKER REGISTRATION: Role=TRUCKER | Cap=32,000 kg | Type=16-Wheel Heavy Trailer => DB Saved [PASS]
- 3. MATCHING ENGINE: Pune -> Kochi (>=20,000 kg) => Found 2 verified truckers with Reefer/Flatbed [PASS]
- 4. SHIPPER RATING: 5-Star Rating & Review => Rating Persisted & User.averageRating updated to 5.0 [PASS]
- 5. TRUCKER PROFILE: Driver Profile Displays => & 5.0 (1 Review), Specs, and Verified Ledger [PASS]

WORKFLOW SUCCESS RATE = 5 / 5 Tested = 100% SUCCESS